# OWASP Juice Shop Web Security Analysis

Dhruv Guda

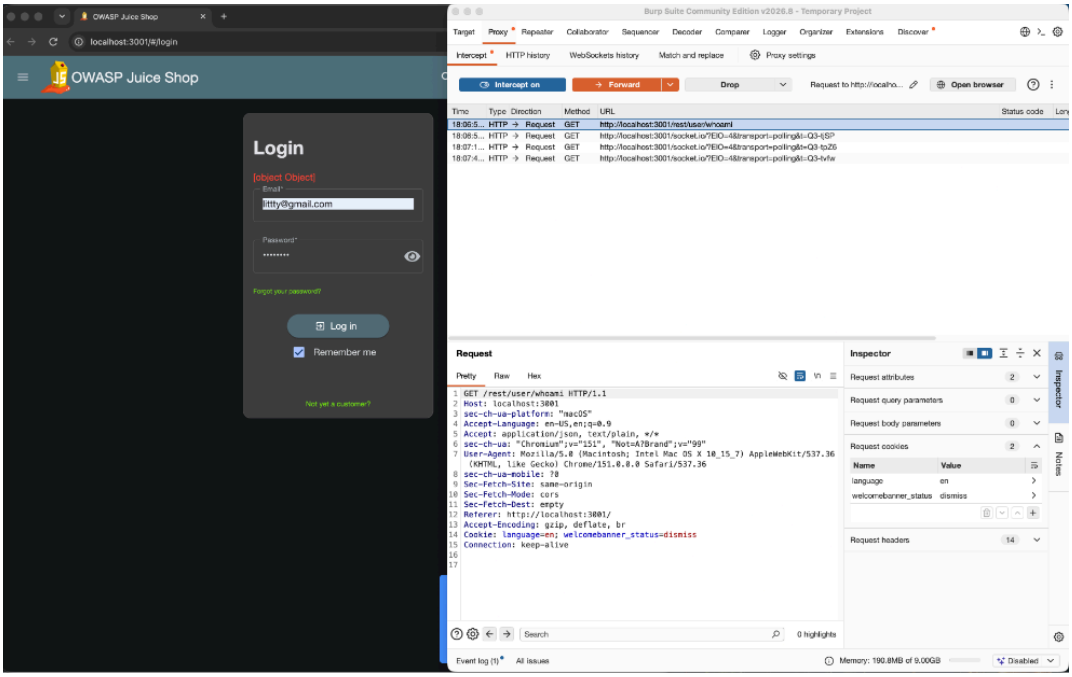
*Original analysis: October 3, 2026; revised October 21, 2026*

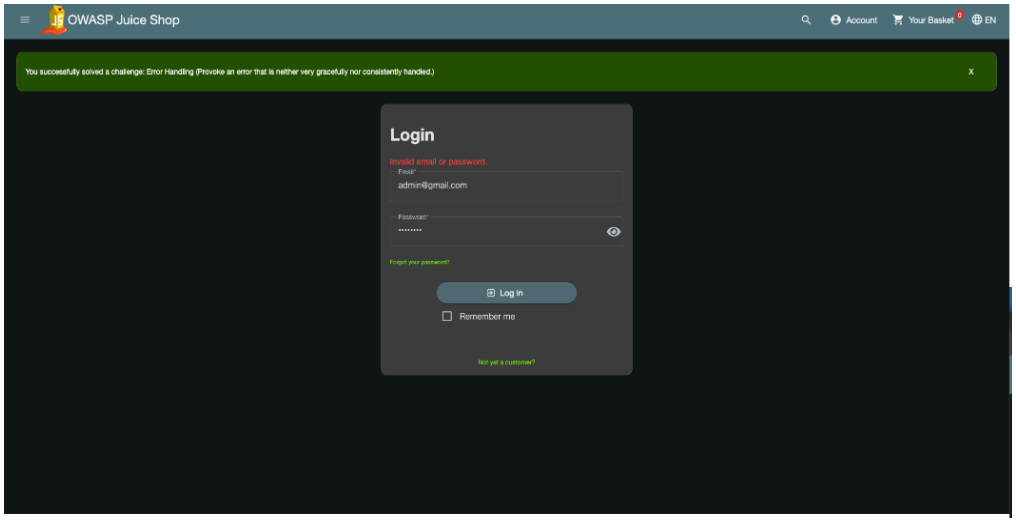
# Introduction

This report examines common web application vulnerabilities using OWASP Juice Shop and a local login lab. It analyzes SQL injection, DOM-based cross-site scripting, exposed credentials, and error handling, explaining their security impact and proposed mitigations (OWASP Foundation, n.d.-c).

Educational lab only. Test exclusively on systems you own or have explicit permission to assess. The included login lab deliberately retains vulnerabilities and is unsuitable for production use. Observed behavior and proposed improvements are distinguished throughout.

## Vulnerability #1: Inconsistent Error Handling






**Error:**

[object Object]

**The message:**

*You successfully solved a challenge:*

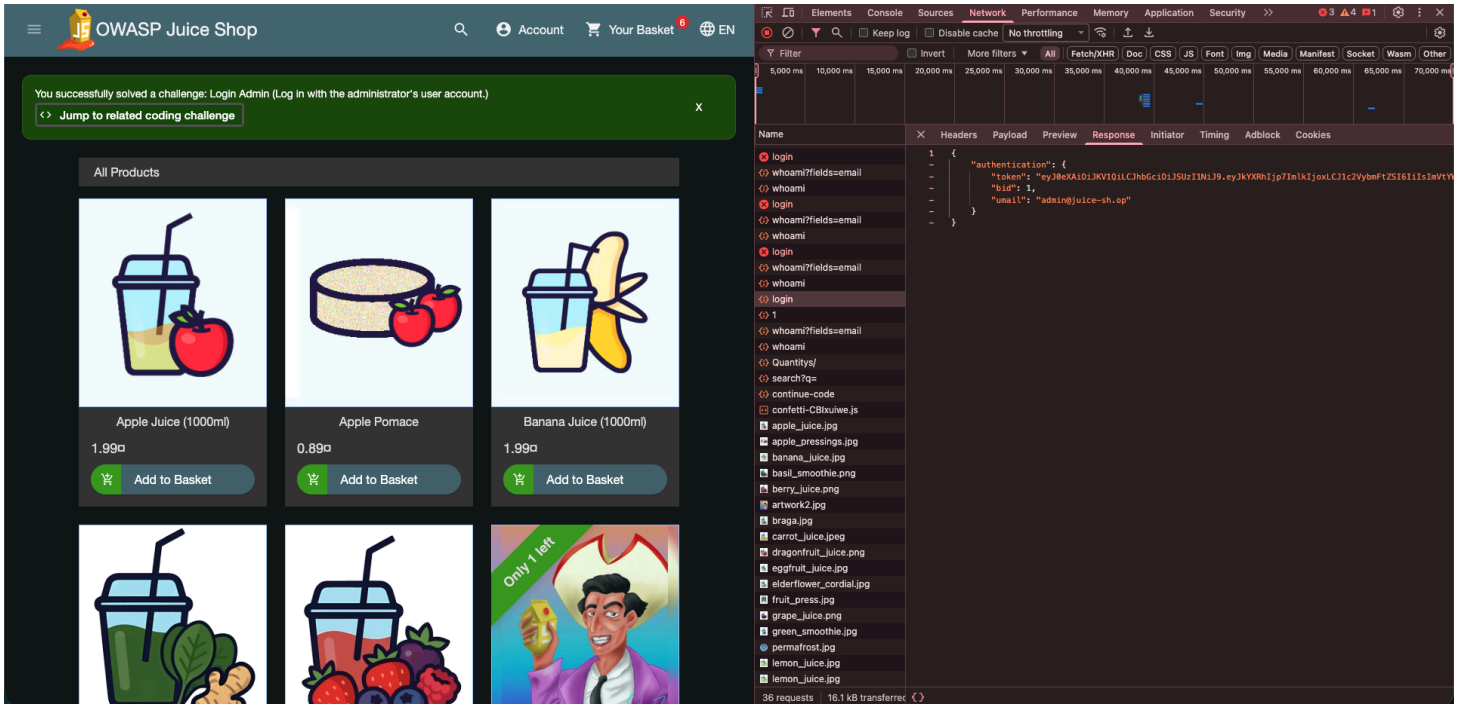
*Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)*

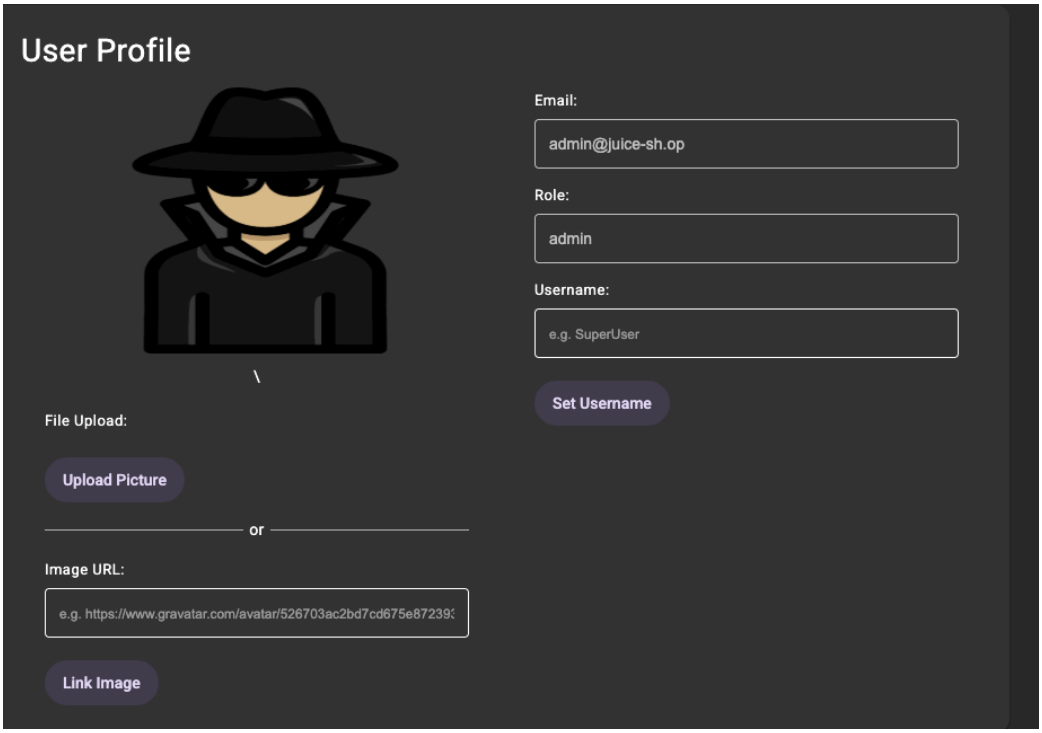
**The how:**

Used Burp Suite to intercept the connection between the application and the server checking for the correct email and password.

Intentionally malformed request body recorded during testing (the exact request endpoint was not independently verified):

```
None
{
"email":"1@"=1"gmail.com",
"Password":"password"
}
```


OWASP Juice Shop
Account
Your Basket
EN
You successfully solved a challenge: Login Admin (Log in with the administrator's user account.)
Jump to related coding challenge
All Products
Apple Juice (1000ml)
1.99¤
Add to Basket
Apple Pomace
0.89¤
Add to Basket
Banana Juice (1000ml)
1.99¤
Add to Basket
Only 1 left
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Headers
Payload
Preview
Response
Initiator
Timing
Adblock
Cookies
"authentication": {
"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoxLCJ1c2VybmFtZSI6IiIsImVtY
"bid": 1,
"umail": "admin@juice-sh.op"
36 requests


User Profile
Email:
admin@juice-sh.op
Role:
admin
Username:
e.g. SuperUser
Set Username
File Upload:
Upload Picture
or
Image URL:
e.g. https://www.gravatar.com/avatar/526703ac2bd7cd675e87239
Link Image

**Mitigation**

Malformed requests should receive consistent, understandable error responses without exposing internal implementation details. Detailed diagnostics should be recorded in protected server-side logs, with sensitive values excluded. These controls are proposed mitigations; their implementation was not verified in this analysis (OWASP Foundation, n.d.-b).

## Vulnerability #2: SQL Injection Authentication Bypass

**The message:**

*You successfully solved a challenge:*

**The how:**

I submitted SQL-shaped input that altered the login query's logic. The observed response identified the administrator account without normal credential matching. The response below is an abbreviated illustration; the authentication token is omitted.

```
None
{
   "authentication": {
      "token":"SOME LARGE HEX VALUE",
      "bid": 1,
      "umail": "admin@juice-sh.op"
   }
}
```

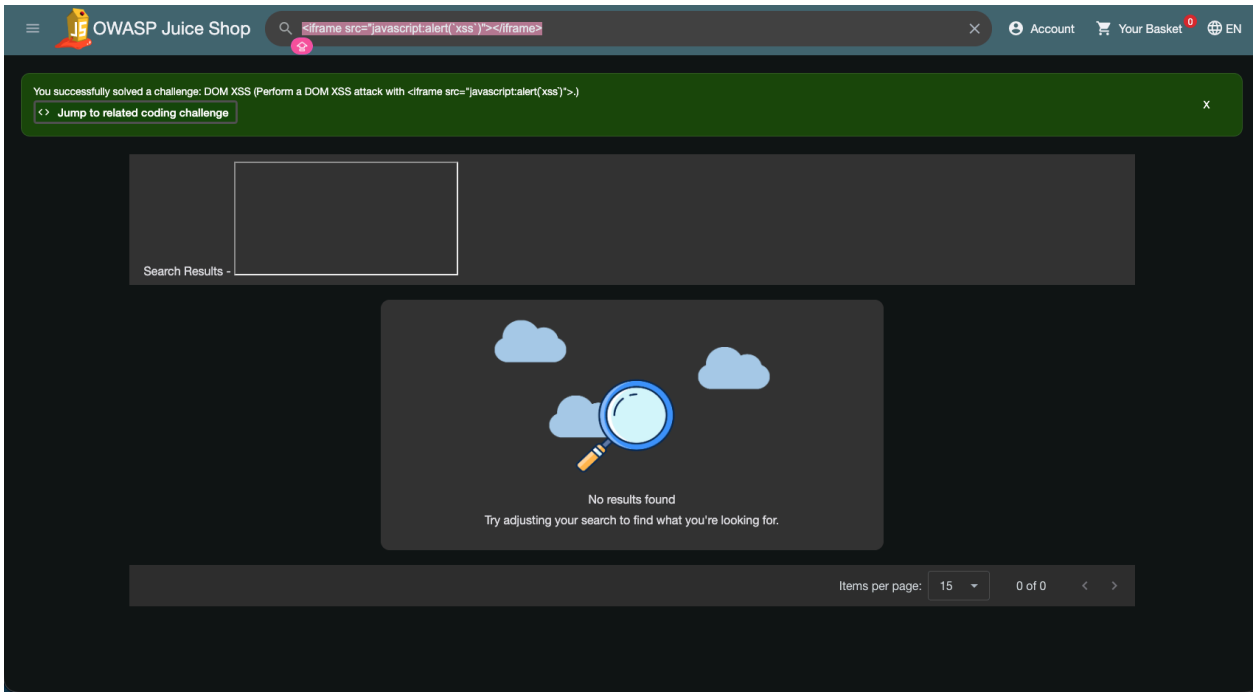
**Mitigation & Explanation:**

Parameterized queries address this injection mechanism by separating submitted values from SQL syntax.

User-supplied quotation marks and SQL operators must remain bound data rather than become part of the SQL statement.

Prepared statements with bound parameters keep submitted values separate from SQL syntax. A string containing quotation marks or SQL operators is treated as a value rather than as an instruction that changes the query. This addresses the demonstrated injection mechanism; it does not, by itself, provide secure password storage or a complete authentication system (OWASP Foundation, n.d.-f).

## Vulnerability #3: DOM-Based XSS




**The message:**

You successfully solved a challenge: DOM XSS (Perform a DOM XSS attack with <iframe src="javascript:alert(`xss`)">.)

**The how:**

Entering <h1> OWASP produced larger text, indicating HTML interpretation but not proving JavaScript execution. I then tested the iframe payload below. The screenshot records the Juice Shop DOM XSS challenge-success banner; this report does not add an independently captured alert result.

<iframe src="javascript:alert(`xss`)"></iframe>

**Mitigation & Explanation:**

For this plain-text search output, render untrusted input as text rather than HTML (OWASP Foundation, n.d.-a).

Use a safe sink such as textContent instead of innerHTML when displaying the search query. If HTML is required, use an appropriate sanitizer and context-specific protections; textContent is not a universal fix for every DOM XSS context.

These are proposed mitigations. Their implementation and effectiveness were not verified in this report.

## Vulnerability #4: Exposed Credentials

**The message:**

You successfully solved a challenge: Exposed Credentials (a developer was careless with hardcoding unused but still valid credentials for a testing account on the client-side.)

**The how:**

To determine if there are any hardcoded passwords in the client-side dev files I initially went through the files trying to find “login” to see if there was a certain function that contained the credentials.

After that I tested “username” to see if there was a variable name with a section of it called username.

The browser-delivered code contained the following public educational credentials supplied by OWASP Juice Shop. They are intentionally exposed lab data, not private production secrets:

```
None
testingUsername = `testing@juice-sh.op`;

testingPassword = `IamUsedForTesting`;
```

Using these credentials, I was able to get into the testing account.

**Mitigation & Explanation:**

Valid credentials must not be embedded in browser-delivered code. If real credentials are exposed, revoke or rotate them and remove them from distributed files and relevant repository history. Server-side secrets should use controlled secret storage, supported by automated secret scanning and code review (OWASP Foundation, n.d.-e).

**Example of secure password handling:**

Passwords should be stored using a purpose-built password-hashing algorithm such as Argon2id, with unique salts and appropriate work parameters. This makes offline guessing more expensive if password hashes are stolen; it does not make passwords impossible to recover. MD5 is unsuitable for production password storage (OWASP Foundation, n.d.-d).

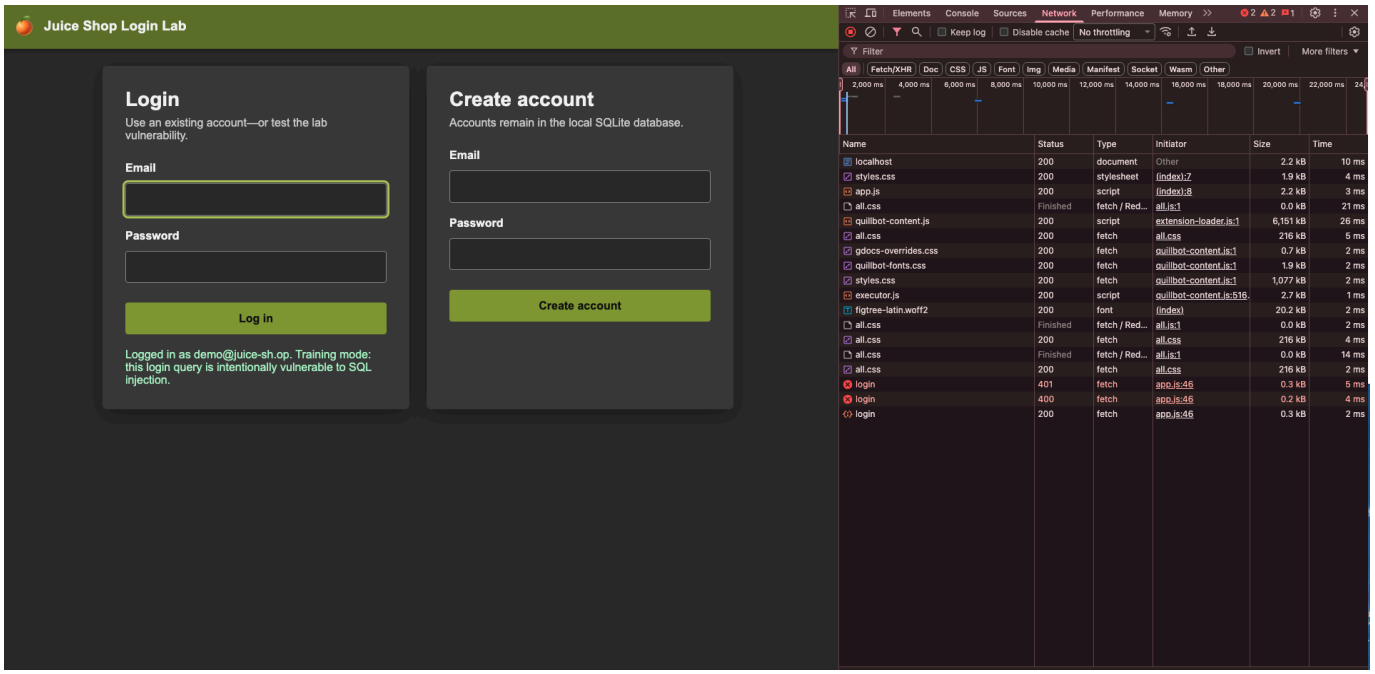
## Part 2: Local Registration and Login Lab

I built a local login lab using HTML, CSS, JavaScript, Node.js, and SQLite, with registration and persistent test accounts. Registration checks for an email containing an @ symbol and a password of at least eight characters; login requires only nonempty input. The lab deliberately uses insecure MD5 password hashing and a SQL-injection-vulnerable login query, allowing an attacker to bypass normal credential checks. These weaknesses support controlled educational testing; a successful login response does not establish an authenticated session.

**GitHub repository:**

https://github.com/d-guda/CS477-Login-Form

# Part 3: Exploit a Vulnerability




Looking at the code, the first vulnerability that came to mind was SQL injection.

A normal login attempt with incorrect credentials returned 401 Unauthorized. For reproducibility, the included local lab's automated test verifies that the exact ASCII email payload ' OR 1=1 -- @ with a nonempty disposable password bypasses the vulnerable query. This is educational test behavior, not a production authentication feature.

**Proposed SQL-Injection Remediation**

One fix is to replace the dynamically constructed SQL query with a parameterized query. Instead of inserting the submitted email directly into the SQL statement, the server should use a placeholder:

```
None
const query = database.prepare(
  'SELECT id, email FROM users WHERE email = ? AND password = ? LIMIT 1'
);

const user = query.get(email, hashPassword(password));
```

Parameterization prevents the demonstrated SQL injection by treating submitted values as data rather than SQL syntax. This proposed change does not address the lab's weak MD5 password hashing or provide a complete authentication system. Password storage would also need an appropriate password-hashing algorithm, such as Argon2id.

## References

OWASP Foundation. (n.d.-a). *DOM based XSS prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *Error handling cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). *OWASP Juice Shop*. https://owasp.org/projects/juice-shop

OWASP Foundation. (n.d.-d). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.-e). *Secrets management cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

OWASP Foundation. (n.d.-f). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html